Go 1.26

The New, The Changed, and The Missing

Agenda

- Optimized
  - cgo
  - garbage collection
  - fmt.Errorf
  - stacked slices
- New
  - new()
  - self-referential generics
  - errors.AsType
  - crypto
- Experimental
  - goroutine leak profile
  - simd/archsimd
  - runtime/secret
- Changed
  - go fix
- Missing
  - JSON v2

CGO

CGO calls are now ~30% faster. Thank you for your attention.

Also, the heap base address is now randomized for added security (64-bit only)

fmt.Errorf

Is now faster for unformatted strings, or strings that only embed %s or %w. This can allow it to be used in more instances, even on a critical path.

Slices

The storage for slices can now be allocated on the stack more frequently, making access faster.

Garbage Collector

Green Tea is now the new default Garbage Collector (GC). It should normally be ~10% faster and situationally up to 40%. It achieves this by handling memory pages instead of single objects in the mark phase of the mark-sweep algorithm used by the GC. This allows for more cpu core memory cache hits and better branch prediction, which more optimally uses the modern CPU architecture.

Fun fact: It is not uncommon that 20% or more of an applications CPU cycles are spent on GC.

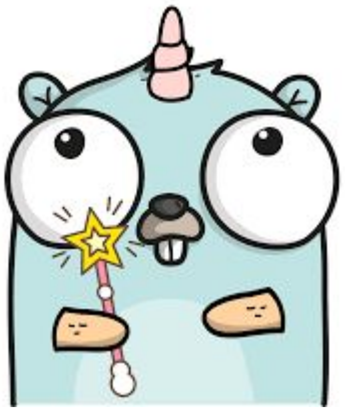
new()

New builtin function `new()` returns a pointer to a newly allocated instance of either the passed type or a pointer to a new variable containing the value passed. The parameter can also be the result of an expression:

```
m := new(MyStruct) // *s => MyStruct{}

b := new(1 == 2) // *b => false

i := new(int64(67)) // *i => 67 (*int64)

t := new(time.Now()) // *t => current time
```

Self-Referential Generics

The restriction that a generic type may not refer to itself in its type parameter list has been lifted.



```
type Adder[A Adder[A]] interface {
    Add(A) A
}

func AddXAndY[A Adder[A]](x, y A) A {
    return x.Add(y)
}
```

errors.AsType

A new generic version of errors.As. A faster, more type-safe, and generally more usable way of getting an error of a specific type from a generic error.

```
_, err := os.Open(“not-a-file”)

pathErr, ok := errors.AsType[*fs.PathError](err)
```

crypto

crypto/hpke

This new package implements the Hybrid Public Key Encryption (HPKE) algorithm, including support for post-quantum hybrid KEMs.

secure random bytes

Many crypto functions now use an internal secure source of random bytes instead of passed random bytes: `crypto/internal/sysrand.Read`

Certain packages ignore passed readers completely. Others only use the internal package if the passed reader is nil.

`testing/cryptotest` has been added to allow for deterministic tests by overriding the global source with the `SetGlobalRandom` function.

go fix

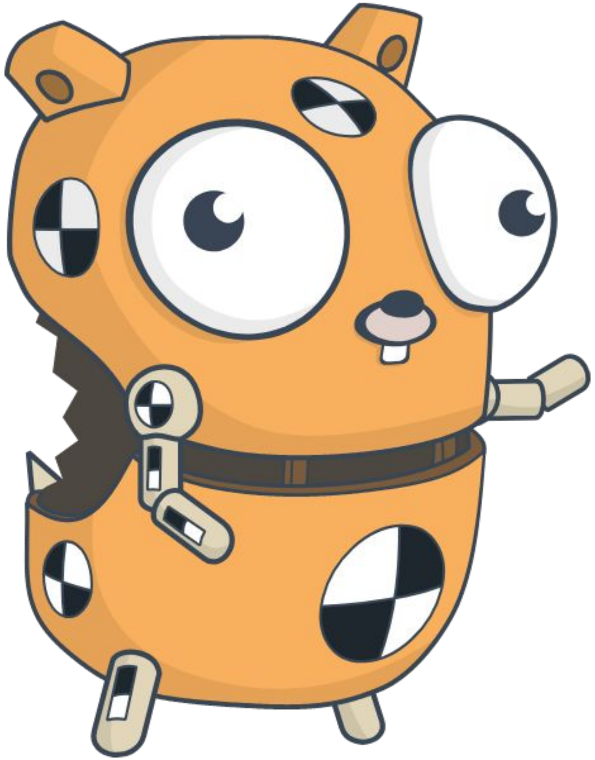
Has been completely revamped to use the same Go analysis framework as `go vet`

Is now home to the modernizers, which can automatically update your code to use more modern features or libraries.

Includes the `//go:fix inline` directive, which allows you to migrate deprecated or changed methods.

First step in an ongoing effort to add an automated migration and update toolchain.

Inclusion of self-developed modernizers and additional directives are being discussed.



Experimental

goroutine leak profile

New pprof profile that can find leaked goroutines. These are goroutines that are blocked by a concurrency primitive, but where the primitive is no longer reachable by any running goroutine.

simd/archsimd

Provides access to Single Input Multiple Data (SIMD) CPU features, and adds new types like Int8x16 and Float64x6 to support 128, 256 and 512 bit vectors.

runtime/secret

“It provides a facility for securely erasing temporaries used in code that manipulates secret information—typically cryptographic in nature—such as registers, stack, new heap allocations. This package is intended to make it easier to ensure forward secrecy. It currently supports the amd64 and arm64 architectures on Linux.”

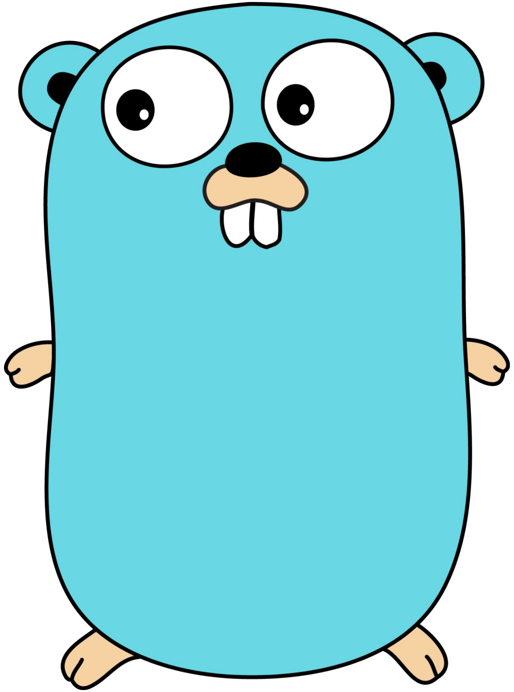
Where’s JSON v2?

- It was decided that the new packages are not quite there, yet. As once out of experimental, it must assure the Go compatibility promise.

- API audit to make sure the new API is “correct”.
- Questions about stricter UTF8 handling and backwards compatibility to JSON v1.
- A certain amount of performance optimization in specific edge-cases (but not in itself a deal-breaker)

Links

- Blogs
  - https://go.dev/blog/go1.26
  - https://go.dev/blog/gofix
  - https://go.dev/blog/greenteagc
- Release Notes
  - https://go.dev/doc/go1.26
  - https://go.dev/doc/devel/release#go1.26.minor
- Examples
  - https://codeberg.org/aeforged/fixme126.git



No-one expects the Bonus Slide! (1.26.1)

- crypto/x509 & net/url
  Security fixes related to how certificates and URLs were parsed
- go fix crash
  Panic when encountering specific complex generic type aliases
- reflect
  Value-to-interface conversion regression that caused slight memory leak